

회원 관리와 DB 기반 인증 구현하기

학습 체크리스트 (1/2)

☐ 회원 엔티티 설계와 JPA Auditing·Lombok 적용 방법 습득

☐ 내부 PK와 외부 식별자를 나누는 이중 식별자 전략 이해

☐ `@ElementCollection` 과 `EnumType.STRING` 으로 권한을 저장하는 방법 파악

☐ 비밀번호 해시의 필요성과 `PasswordEncoder` 동작 이해

학습 체크리스트 (2/2)

☐ UserDetails · UserDetailsService 구현으로 DB 인증 연결

☐ 회원 가입 요청의 검증·중복 검사·인코딩 흐름 습득

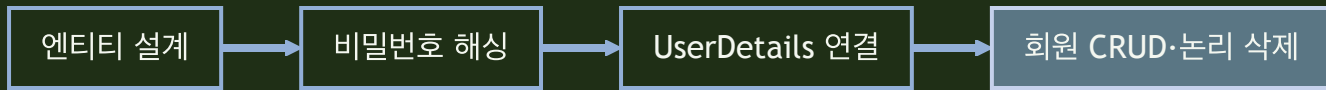
☐ 논리 삭제(Soft Delete)로 탈퇴를 처리하는 방법 파악

☐ 회원 경로별 인가 규칙과 응답 DTO 분리 원칙 확인

이번 차시 범위

- 출발점: `application.yml` 인메모리 계정 1개 → DB 기반 회원으로 전환
- 다룰 기능: 회원 가입, 조회, 수정, 탈퇴(논리 삭제)
- 제외 범위: JWT, OAuth2, 이메일 인증은 별도 차시
- 전제: 인증·인가 필터 체인 구조는 이미 학습 완료

이번 차시 진행 순서



엔티티 이름을 UserAccount로 정하는 이유

후보명	문제점
User	Spring Security의 <code>org.springframework.security.core.userdetails.User</code> 와 이름 충돌
user	MySQL 예약어와 충돌해 테이블명 사용 시 이스케이프 필요
UserAccount	위 두 충돌을 모두 회피하고 의미도 명확

생성·수정 시각 공통 처리 베이스 엔티티

```
1  @MappedSuperclass
2  @EntityListeners(AuditingEntityListener.class)
3  public abstract class BaseTimeEntity {
4
5      @CreatedDate
6      private Instant createdAt;
7
8      @LastModifiedDate
9      private Instant updatedAt;
10 }
```

JPA Auditing이 대신 해주는 일

- 자동 기록: `@CreatedDate` · `@LastModifiedDate` 필드가 저장·수정 시점에 자동으로 채워짐
- 수동 코드 불필요: `Instant.now()` 를 직접 호출해 시각을 채우지 않아도 됨
- 리스너만으론 부족: `@EntityListeners` 선언만으로는 동작하지 않음
- 활성화 필수: 애플리케이션 설정에 Auditing 기능을 별도로 켜야 함

JPA Auditing 기능 활성화

```
1 @Configuration
2 @EnableJpaAuditing
3 class JpaAuditingConfig {
4 }
```

UserAccount 엔티티 필드 구성

```
1  @Entity @Getter
2  public class UserAccount extends BaseTimeEntity {
3      @Id @GeneratedValue
4      private Long id;
5      private UUID uuid;
6      private String username;
7      private String password;
8      private String nickname;
9      private boolean enabled; // roles는 뒤에서 추가
10 }
```

Lombok 어노테이션의 역할

어노테이션	역할
<code>@Getter</code>	필드 조회 메서드만 공개, 세터는 생성하지 않음
<code>@NoArgsConstructor(PROTECTED)</code>	JPA 프록시 생성용 기본 생성자, 외부 무분별 생성 차단
<code>@AllArgsConstructor(PRIVATE)</code> + <code>@Builder</code>	빌더를 통한 생성만 허용
<code>@Builder.Default</code>	빌더에서 값을 지정하지 않은 필드의 기본값 유지

정적 팩토리로 회원 생성·탈퇴 처리

```
1  public static UserAccount register(String username, String encodedPassword, String nickname) {
2      return UserAccount.builder()
3          .username(username).password(encodedPassword)
4          .nickname(nickname).enabled(true)
5          .build();
6  }
7  public void withdraw() {
8      this.enabled = false;
9  }
```

엔티티가 지키는 불변식

- **세터 미제공**: 상태 변경은 `register` · `withdraw` 처럼 의도가 드러나는 메서드로만 허용
- `encodedPassword` **네이밍**: 파라미터명 자체로 이미 인코딩된 값을 명시해 평문 전달 실수 방지
- `getRoles()` **반환**: 내부 컬렉션을 그대로 주지 않고 `unmodifiableSet` 으로 감싸 반환
- **목적**: 엔티티 내부 상태 규칙을 코드 레벨에서 강제

이중 식별자 전략

구분	Long id	UUID uuid
용도	내부 PK, 조인·인덱스 성능	외부 노출용 식별자
특징	순차 증가, 값 추정이 쉬움	무작위값, 식별자 열거 난이도 ↑
노출 범위	API·URL에 노출 금지	API·URL에 노출

- 주의: UUID 사용이 인가를 대체하지 않음, 객체 단위 인가 검사는 별도로 필수

Role enum 선언

```
1  public enum Role {  
2      USER, ADMIN;  
3  
4      public String key() {  
5          return "ROLE_" + name();  
6      }  
7  }
```

hasRole와 ROLE_ 접두사 규약

- `hasRole("ADMIN")`: 내부적으로 `ROLE_ADMIN` 권한 보유 여부를 검사
- `getAuthorities()`: 반환되는 권한 문자열에는 항상 접두사가 붙어 있어야 함
- enum 값 자체: `ADMIN` 처럼 접두사 없이 순수한 이름만 유지
- 접두사 부착 시점: `key()` 호출 지점에서만 `"ROLE_"` 을 붙여 변환

roles 필드 매핑

```
1  @ElementCollection(fetch = FetchType.EAGER)
2  @CollectionTable(
3      name = "user_account_roles",
4      joinColumns = @JoinColumn(name = "user_account_id")
5  )
6  @Enumerated(EnumType.STRING)
7  private Set<Role> roles = new HashSet<>();
```

roles 필드에 적용된 매핑 전략

- `@ElementCollection`: 엔티티가 아닌 값 타입 컬렉션 매핑, 별도 리포지토리 불필요
- **생명주기 종속**: roles는 UserAccount 없이 독립적으로 존재할 수 없음
- `FetchType.EAGER`: 로그인·권한 검사 시 항상 필요해 즉시 로딩
- `LazyInitializationException` 방지: 영속성 컨텍스트가 닫힌 뒤 접근해도 안전
- `@CollectionTable`: RDB는 한 컬럼에 컬렉션을 저장할 수 없어 보조 테이블로 분리

@Enumerated: STRING vs ORDINAL

저장 방식	저장값	위험 요소
STRING	"ADMIN", "USER" 문자열	enum 순서가 바뀌어도 안전
ORDINAL	0, 1 숫자 인덱스	enum 순서 변경 시 권한이 조용히 뒤바뀜

- 디버깅 가독성: STRING은 DB 값만 보고도 권한을 바로 파악 가능

UserAccountRepository

```
1 public interface UserAccountRepository
2     extends JpaRepository<UserAccount, Long> {
3
4     Optional<UserAccount> findByUsername(String username);
5     Optional<UserAccount> findByUuid(UUID uuid);
6     boolean existsByUsername(String username);
7 }
```

중복 아이디를 막는 이중 방어선

- 1차 방어: 서비스 계층에서 `existsByUsername` 으로 사전 검사
- 2차 방어: DB `unique` 제약이 경합 상황의 최종 방어선
- 경합 상황: 동시 요청 시 사전 검사를 통과해도 커밋 시점에 충돌 가능
- 경계 확인: `saveAndFlush()` 또는 `flush()` 로 예외를 트랜잭션 경계 안에서 확인

비밀번호는 암호화가 아니라 해시로

해시 (Hash)

원문을 복원할 수 없는 단방향 변환, 검증만 가능하고 복호화는 불가능

- 양방향 암호화 금지: 키가 유출되면 모든 비밀번호가 그대로 노출됨
- salt: 같은 비밀번호라도 저장값이 달라지도록 무작위 값을 섞음
- 레인보우 테이블 방어: salt가 없으면 미리 계산된 해시 표로 역추적 가능
- 범용 해시 배제: MD5·SHA-256은 빠른 게 목적이라 비밀번호 저장에 부적합

bcrypt · scrypt · Argon2 비교

알고리즘	비용 방식	한계
bcrypt	CPU 연산 반복(work factor)	메모리 사용이 적어 GPU 병렬화에 취약
scrypt	대용량 메모리 요구로 GPU 방어	파라미터 튜닝이 까다로움
Argon2id	시간·메모리·병렬성 3차원 조절	현재 권장 표준

- **작업 비용 인자(work factor)**: 전용 해시는 의도적으로 느려 GPU·ASIC 병렬 대입 비용을 키움
- **흐름**: CPU 지연(bcrypt) → 메모리 장벽(scrypt) → 시간·메모리·병렬성 3차원(Argon2id)

PasswordEncoder 빈 등록

```
1  @Bean
2  PasswordEncoder passwordEncoder() {
3      return PasswordEncoderFactories.createDelegatingPasswordEncoder();
4  }
```

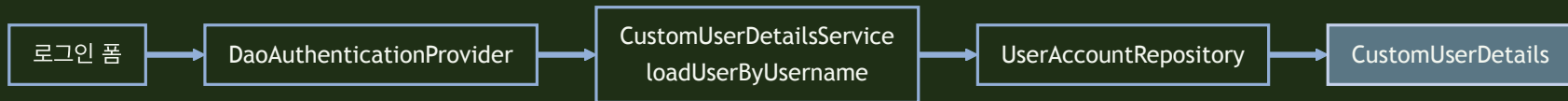

DelegatingPasswordEncoder가 하는 일

- {bcrypt} 접두사: 저장된 해시 앞에 사용된 알고리즘을 기록
- 알고리즘 점진 전환: `upgradeEncoding` 으로 새 알고리즘으로 서서히 교체 가능
- `encode()` 호출 시점: 회원가입·비밀번호 변경 시 우리가 직접 호출
- `matches()` 호출 시점: 로그인 검증 시 `DaoAuthenticationProvider` 가 내부에서 호출

비밀번호를 다룰 때 지켜야 할 것

- 로그 금지: 로그 메시지에 비밀번호 값을 남기지 않음
- toString() 제외: 엔티티의 `toString()` 에 비밀번호 필드 포함 금지
- 응답 DTO 제외: 클라이언트로 보내는 응답에 비밀번호를 절대 담지 않음
- 원칙: 저장 후에는 원문이 어디에도 남지 않아야 함

로그인 요청이 DB 회원에 닿기까지



UserDetails가 표현하는 두 가지 정보

메서드	구분	의미
<code>getUsername()</code>	신원	로그인 아이디 반환
<code>getPassword()</code>	신원	인코딩된 비밀번호 반환
<code>getAuthorities()</code>	신원	보유 권한 목록 반환
<code>isAccountNonExpired()</code>	상태	계정 기간이 만료되지 않았는지
<code>isAccountNonLocked()</code>	상태	계정이 잠기지 않았는지
<code>isCredentialsNonExpired()</code>	상태	비밀번호가 만료되지 않았는지
<code>isEnabled()</code>	상태	계정이 활성화 상태인지

계정 상태 메서드와 연결된 예외

예외	관련 메서드	이번 도메인
<code>AccountExpiredException</code>	<code>isAccountNonExpired()</code>	항상 <code>true</code>
<code>LockedException</code>	<code>isAccountNonLocked()</code>	항상 <code>true</code>
<code>CredentialsExpiredException</code>	<code>isCredentialsNonExpired()</code>	항상 <code>true</code>
<code>DisabledException</code>	<code>isEnabled()</code>	<code>UserAccount.enabled</code> 반영

- `isEnabled()` 만 실제 값을 반영하며, 탈퇴(논리 삭제) 시 `false` 로 전환되어 로그인 차단

CustomUserDetails 필드 선언

```
1 public class CustomUserDetails implements UserDetails {  
2  
3     private final Long id;  
4     private final String username;  
5     private final String password;  
6     private final String nickname;  
7     private final boolean enabled;  
8     private final List<SimpleGrantedAuthority> authorities;  
9 }
```

UserAccount 값을 복사해 권한으로 변환

```
1 public CustomUserDetails(UserAccount account) {  
2     this.id = account.getId();  
3     this.username = account.getUsername();  
4     this.password = account.getPassword();  
5     // nickname, enabled 복사 생략 (Getter도 동일)  
6     this.authorities = account.getRoles().stream()  
7         .map(role → new SimpleGrantedAuthority(role.key())) .toList();  
8 }
```

엔티티가 아닌 래퍼 클래스를 쓰는 이유

- API 결합 방지: 도메인 엔티티(`UserAccount`)가 Spring Security API에 직접 의존하지 않도록 분리
- 세션 직렬화 대상: `UserDetails`는 로그인 성공 후 세션에 그대로 저장되는 객체
- 값만 복사: 엔티티 참조 대신 필요한 값만 꺼내 담아 세션에 안전하게 보관
- `LazyInitializationException` 차단: 세션 역직렬화 시점에 지연 로딩 프록시가 끊겨 발생하는 예외를 원천 방지

아이디로 계정을 조회하는 서비스

```
1  @Service
2  @RequiredArgsConstructor
3  public class CustomUserDetailsService implements UserDetailsService {
4      private final UserRepository repository;
5
6      @Transactional(readOnly = true)
7      public UserDetails loadUserByUsername(String username) {
8          return repository.findByUsername(username).map(CustomUserDetails::new)
9              .orElseThrow(() -> new UsernameNotFoundException(username));
10     }
11 }
```

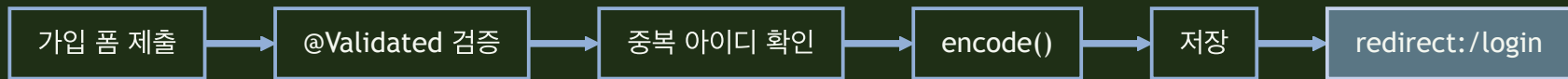
빈 등록만으로 인증에 연결되는 이유

- `readOnly = true`: 조회만 수행하는 트랜잭션이므로 변경 감지를 생략해 성능 이점 확보
- 자동 구성 후퇴: 이 빈을 등록하는 순간 `@ConditionalOnMissingBean`에 의해 `spring.security.user.*` 인메모리 자동 구성이 물러남
- 자동 탐색: `DaoAuthenticationProvider`가 컨텍스트의 `UserDetailsService`와 `PasswordEncoder`를 스스로 찾아 사용
- 연결 코드 불필요: `SecurityConfig`에 별도로 이어주는 코드를 작성할 필요 없음

계정 조회 실패 메시지를 감추는 이유

- 직접 노출 금지: `UsernameNotFoundException` 메시지를 화면에 그대로 보여주면 "아이디 없음"이 드러남
- `hideUserNotFoundExceptions` (기본 `true`): `DaoAuthenticationProvider`가 이 예외를 `BadCredentialsException`으로 감싸서 전달
- 동일 실패 처리: 아이디 오류와 비밀번호 오류가 서버 내부적으로 같은 예외로 통일됨
- 끄지 않기: 이 옵션을 끄면 계정 열거(user enumeration) 공격에 악용될 수 있음

회원가입 요청 처리 흐름



SignupRequest 검증 규칙

```
1  public record SignupRequest(  
2      @NotBlank @Size(min = 4, max = 20) String username,  
3      @NotBlank @Size(min = 8, max = 64) String password,  
4      @NotBlank @Size(min = 2, max = 20) String nickname  
5  ) {  
6  }
```

가입 폼 제출과 재검증 분기

```
1  @GetMapping("/signup")
2  String form(Model model) {
3      model.addAttribute("signupRequest", new SignupRequest("", "", ""));
4      return "signup";
5  }
6  @PostMapping("/signup")
7  String submit(@Validated SignupRequest req, BindingResult br) {
8      if (br.hasErrors()) return "signup";
9      userService.signup(req); return "redirect:/login?signup";
10 }
```

가입 서비스의 중복 검사와 인코딩

```
1  @Transactional
2  public void signup(SignupRequest request) {
3      if (userRepository.existsByUsername(request.username())) {
4          throw new DuplicateUsernameException(request.username());
5      }
6      String encoded = passwordEncoder.encode(request.password());
7      UserAccount account = UserAccount.register(
8          request.username(), encoded, request.nickname());
9      userRepository.save(account);
10 }
```

가입 폼 필드와 오류 표시

```
1 <form th:action="@{/signup}" th:object="${signupRequest}" method="post">
2   <input type="text" th:field="*{username}" />
3   <p th:if="${#fields.hasErrors('username')}" th:errors="*{username}"></p>
4   <!-- password, nickname 필드도 동일 패턴 -->
5   <button type="submit">가입하기</button>
6 </form>
```


폼 재표시와 CSRF 토큰

- `th:action` 필수: CSRF 토큰 hidden 필드가 자동 삽입, 순수 `action` 은 403
- 재표시 대상: 검증 실패 시 아이디·닉네임은 `th:field` 가 기존 값을 유지
- 비밀번호 예외: `type="password"` 는 브라우저가 값을 비워 다시 입력 필요
- 원인: 재표시되는 값은 서버가 되돌려준 DTO 값뿐, 비밀번호는 저장하지 않음

조회와 수정 정책

기능	경로/조건
본인 조회	<code>GET /user-accounts/me</code>
관리자 전체 조회	<code>GET /admin/user-accounts</code>
닉네임 변경	새 닉네임 값만 검증 후 반영
비밀번호 변경	<code>passwordEncoder.matches(현재 비밀번호, ...)</code> 선확인 + 다른 세션 무효화

물리 삭제 대신 논리 삭제

방식	동작	위험
물리 삭제	<code>DELETE</code> 로 행 자체를 제거	이력·연관 데이터 복구 불가
논리 삭제	<code>enabled=false</code> 로 상태만 변경	조회 조건 누락 시 탈퇴 계정 노출

- 구현 ①: `withdraw()`로 `enabled` 변경 → Dirty Checking, 로그인 시 `DisabledException`
- 구현 ②: `@SQLDelete` + `@SQLRestriction`으로 자동 가로채기 → 조회에서 감춰져 `UsernameNotFoundException`
- 주의: `username` unique 제약과 충돌 → 탈퇴 시 아이디 가공 필요(NULL 다중 허용 DB에서 복합 유니크는 무효)

탈퇴 처리와 세션 종료

```
1  @PostMapping("/user-accounts/me/withdraw")
2  String withdraw(@AuthenticationPrincipal CustomUserDetails principal,
3                  HttpServletRequest request, HttpServletResponse response) {
4      userService.withdraw(principal.getId());
5      Authentication auth = SecurityContextHolder.getContext().getAuthentication();
6      new SecurityContextLogoutHandler().logout(request, response, auth);
7      return "redirect:/login?withdrawn";
8  }
```

응답 전용 UserAccountResponse

```
1  public record UserAccountResponse(  
2      UUID uuid, String username, String nickname, Set<Role> roles) {  
3  
4      public static UserAccountResponse from(UserAccount account) {  
5          return new UserAccountResponse(  
6              account.getUuid(), account.getUsername(),  
7              account.getNickname(), account.getRoles());  
8      }  
9  }
```

회원 경로별 인가 규칙

경로	규칙
<code>/signup</code>	<code>permitAll()</code>
<code>/user-accounts/**</code>	<code>authenticated()</code>
<code>/admin/**</code>	<code>hasRole("ADMIN")</code>

- 인코딩·기본 권한: 비밀번호 인코딩과 `Role.USER` 부여는 `register()` 내부, 가입 경로 하나로 단일화
- 모순 방지: `/signup` 이 `permitAll` 이 아니면 "계정이 있어야 가입 가능"한 모순 발생
- 자원별 세부 권한(작성자 본인만 수정)은 다음 RBAC 절에서 다룸

학습 요약 (1/2)

- 엔티티 설계: `UserAccount`로 이름 충돌을 피하고, 상태 변경은 의도가 드러나는 메서드로만
- 공통 감사 정보: `BaseTimeEntity` + `@EnableJpaAuditing`으로 생성·수정 시각 자동 기록
- 이중 식별자: 내부는 `Long id`, 외부 노출은 `UUID uuid`, 인가 검사는 별도로 필수
- 권한 저장: `@ElementCollection` 값 타입 + `EnumType.STRING`, `ROLE_` 접두사는 `key()`에서만

학습 요약 (2/2)

- 비밀번호: 항상 `PasswordEncoder` 를 거쳐 저장, 로그 · `toString()` · 응답에 절대 노출 금지
- 인증 연결: `CustomUserDetails` 에 값만 복사하고 `CustomUserDetailsService` 로 DB 조회
- 회원 가입: 사전 중복 검사 + DB unique 제약, 인코딩은 서비스의 단일 경로에서
- 탈퇴 처리: 참조 무결성을 지키는 논리 삭제와 인증 세션 정리를 함께 수행